

F-01

## Propuesta técnica preliminar

Decisiones de arquitectura, tecnología y despliegue, con su justificación. Se aprueba antes de escribir código.

► Se diligencia en la semana 3. Toda decisión debe estar justificada: no basta con nombrar la tecnología, hay que explicar por qué es la adecuada para este problema y qué alternativa se descartó.

### 1. Datos generales

|                     |                                                      |
|---------------------|------------------------------------------------------|
| Proyecto            | Sistema Interno de Aseguramiento de la Calidad SIAC  |
| Dependencia cliente | Planeación                                           |
| Integrantes         | David Urrutia Ceron - Deiby Alejandro Ramirez Galvis |
| Fecha               | 20 Agosto del 2026                                   |

### 2. Arquitectura propuesta

#### 2.1 Estilo arquitectónico

Este proyecto se desarrolla bajo una arquitectura **cliente-servidor orientada a servicios**: un backend que expone una API consumida por un cliente web y un cliente móvil. Describa cómo se materializa en su caso.

Este proyecto se desarrollará bajo una arquitectura **cliente-servidor monolítica en capas**, donde frontend y backend están separados y se comunican a través de una **API REST**. Esta separación es intencional y forma parte de la estrategia de diseño.

#### Distribución de Capas

##### En el Frontend (Next.js):

1. **Capa de Presentación:** Son componentes React, páginas, layouts, estilos (Tailwind CSS). Renderizado en el servidor (SSR) para mejorar el rendimiento.
2. **Capa de Cliente API:** Los servicios que realizan peticiones HTTP al backend (usando **fetch** o librerías como **react-query**). Manejan el estado de las consultas y la caché del lado del cliente.

**En el Backend (NestJS - Monolito):**

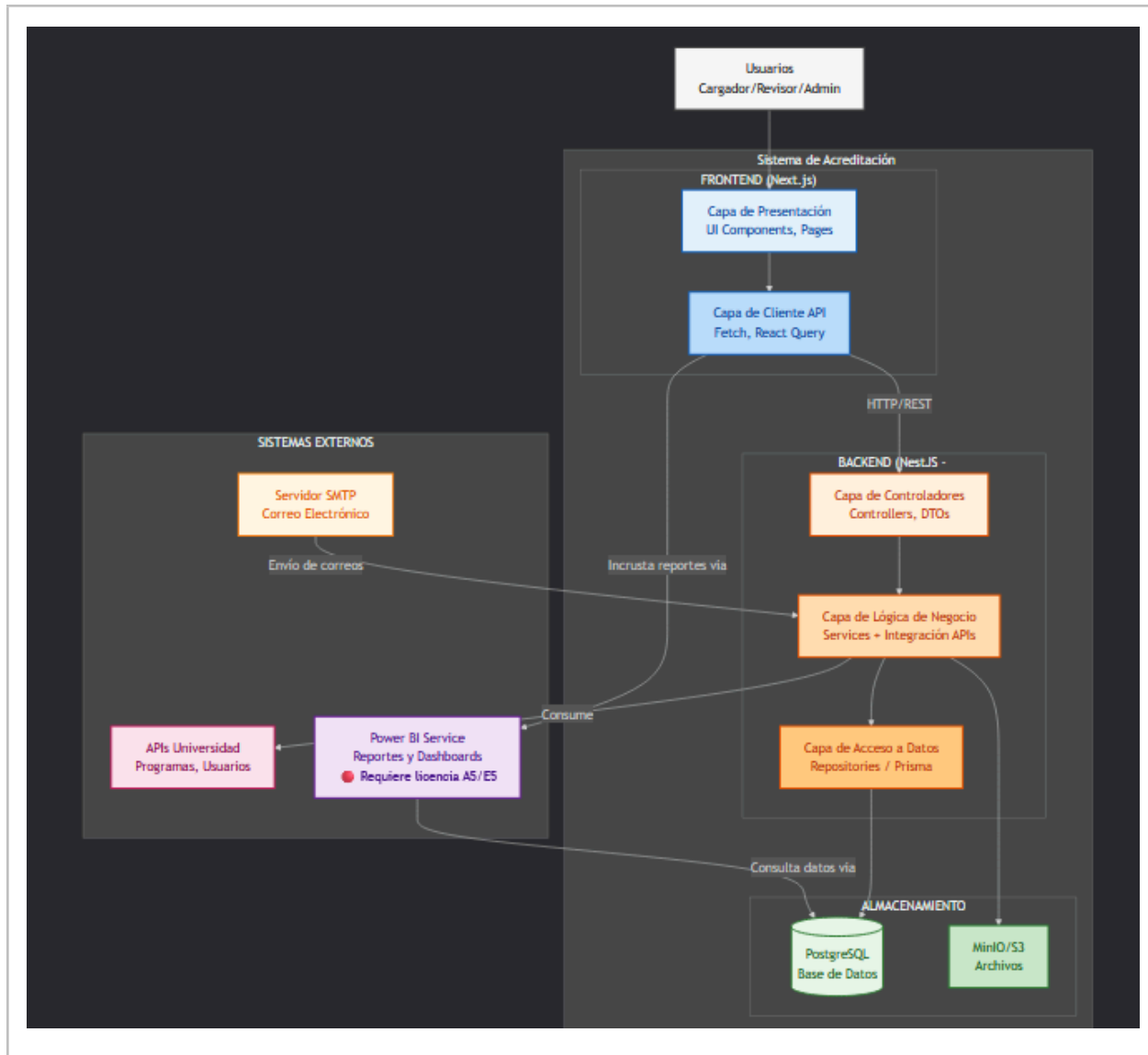
1. **Capa de Controladores (Controllers):** Reciben las peticiones HTTP, validan los datos de entrada (usando DTOs y pipes de validación) y retornan las respuestas en formato JSON.
2. **Capa de Lógica de Negocio (Services):** Contiene toda la lógica central del sistema:
  - Validación de roles y permisos
  - Control de vigencias de documentos
  - Flujo de aprobación
  - Parseo de archivos Excel
  - Generación de alertas y notificaciones
  - **Servicios de Integración:** Módulo especializado que consume las APIs externas de la universidad, actuando como adaptadores.
3. **Capa de Acceso a Datos (Repositories):** Utiliza Prisma ORM para interactuar con la base de datos. Cada repositorio encapsula las operaciones CRUD para una entidad específica.

**En el Almacenamiento (Separado del Backend):**

1. **Base de Datos (PostgreSQL):** Almacena datos estructurados.
2. **Almacenamiento de Archivos (S3/MinIO):** Almacena archivos físicos (PDFs, Excels).

## 2.2 Diagrama de contexto

Inserte el diagrama que muestre el sistema, sus clientes y los sistemas externos con los que se integra.



## 2.3 Justificación

¿Por qué esta arquitectura y no otra? ¿Qué alternativa evaluaron y por qué la descartaron?

Realizamos una evaluación de tres estilos arquitectónicos principales:

1. **Arquitectura de Microservicios:** Dividir el sistema en pequeños servicios independientes.
  - **Descartada porque:** La falta de experiencia del equipo, el plazo ajustado (2.5 meses), la complejidad operativa y el hecho de que las APIs externas son dependencias, no servicios propios, hacen que esta arquitectura sea inadecuada. Sería un sobrediseño que pondría en riesgo la entrega del producto.
2. **Arquitectura Hexagonal o de Puertos y Adaptadores:** Un estilo que busca aislar el dominio de la aplicación de los detalles externos.

- **Descartada porque:** Si bien es una excelente arquitectura para sistemas con reglas de negocio muy complejas, añade una capa de abstracción adicional que, para nuestro caso, no es necesaria en esta fase. El equipo tiene experiencia con el patrón de capas, y es más directo de implementar en el plazo disponible.
3. **Monolito Tradicional (Frontend y Backend Juntos):** Todo el código en una sola aplicación (ej. Django con templates).
- **Descartada porque:** No permite la separación clara de responsabilidades, dificulta el despliegue independiente y limita la reutilización de la API para futuros clientes (ej. aplicación móvil).

### 3. Stack tecnológico

| Capa          | Tecnología / framework | Versión | Justificación                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------|------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend / API | Nest.js                | v10.x   | Elegimos NestJS porque es un framework de Node.js que impone una estructura clara y modular (similar a Angular), lo que facilita la organización del código, las pruebas y el mantenimiento. Su soporte nativo para Inyección de Dependencias y la creación de módulos, controladores y servicios se alinea perfectamente con la complejidad del sistema. <b>Alternativa descartada:</b> Express.js. Si bien es más ligero y el equipo lo conoce, no ofrece una estructura por defecto, lo que en un proyecto con múltiples roles, flujos y validaciones podría llevar a un código menos mantenible y más difícil de escalar. |

|                  |                  |      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------|------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Base de datos    | Postgres         | 14.x | Es un sistema de base de datos relacional maduro, robusto y de código abierto. Es la opción ideal para un sistema con datos altamente estructurados (indicadores, evidencias, programas, roles) y con relaciones complejas. Su soporte nativo para búsqueda de texto completo (Full-Text Search) es un requisito clave del motor de búsqueda. Además, su compatibilidad con Supabase y con el servidor institucional facilita la migración planificada.                                                                                    |
| Frontend web     | React            | 14.x | Next.js proporciona una estructura sobre React que ofrece renderizado del lado del servidor (SSR), generando páginas completas en el servidor. Esto es más adecuado para una aplicación web tradicional con apartados y dashboards, ya que mejora los tiempos de carga inicial y facilita la gestión del estado de la navegación. <b>Alternativa descartada:</b> Create React App (CRA). Si bien es una SPA, no ofrece SSR y el manejo del estado en la URL y el enrutamiento es menos eficiente para una aplicación con múltiples vistas. |
| Aplicación móvil | No Aplica        | N/A  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Autenticación    | JWT + OAuth2 2.0 | N/A  | Se usa JWT (JSON Web Tokens) para manejar la sesión de forma stateless, lo cual es ideal para una API REST. Se implementará con la librería Passport.js en NestJS, que soporta múltiples estrategias. Se usará un flujo de "password" para el login inicial. La integración con OAuth2 se deja como una mejora futura, pero no es crítica para el MVP. Es una práctica estándar y segura que el equipo conoce.                                                                                                                             |

|                      |                                                   |     |                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------|---------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pruebas              | Jest (Backend) + React Testing Library (Frontend) | N/A | Jest es el framework de pruebas más popular y maduro para JavaScript/TypeScript, con una excelente integración con NestJS y React. React Testing Library es el estándar para pruebas de componentes React, ya que promueve pruebas que se asemejan a la interacción del usuario. Esta combinación es conocida por el equipo y cubre las necesidades de pruebas unitarias y de integración. |
| Control de versiones | Git Hub                                           | N/A | Se usará GitHub por su integración con el ecosistema de desarrollo y porque el repositorio es requerido por el docente para evaluar la participación.                                                                                                                                                                                                                                      |
| Gestión del proyecto | Clickup                                           | N/A | Elegimos Trello por su simplicidad y flexibilidad para equipos pequeños. Nos permite gestionar el backlog, las tareas del sprint y el progreso de una manera visual y ágil, que es la metodología que seguiremos (Scrum). Alternativa descartada: Jira, que es más potente pero demasiado compleja para un equipo de dos personas en un proyecto de 2.5 meses.                             |

► La columna de justificación es la que se evalúa. «Porque lo vimos en clase» no es una justificación técnica; «porque el equipo domina el lenguaje y la carga esperada no exige más» sí lo es.

## 4. Patrones de diseño y arquitectura

Indique los patrones que aplicarán, dónde y para resolver qué problema.

| Patrón                  | Dónde se aplica         | Problema que resuelve                                                                                                                                                                                                                                                                |
|-------------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Módulos (NestJS)</b> | En la capa del Backend. | Organiza el código en módulos cohesivos (ej. <code>AuthModule</code> , <code>DocumentsModule</code> , <code>ProgramsModule</code> ), cada uno con sus propios controladores, servicios y entidades. Esto resuelve el problema de la organización del código en un sistema con muchas |

| Patrón                            | Dónde se aplica                                                                 | Problema que resuelve                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                   |                                                                                 | funcionalidades, facilitando el mantenimiento y la escalabilidad.                                                                                                                                                                                                                                                                                                                             |
| <b>Repository</b>                 | En la capa de acceso a datos (Backend).                                         | Actúa como una capa de abstracción entre la lógica de negocio y el ORM (Prisma). Cada repositorio (ej. <b>ProgramRepository</b> ) encapsula todas las operaciones de base de datos para una entidad, permitiendo cambiar de ORM o de motor de base de datos en el futuro sin afectar al resto de la aplicación.                                                                               |
| <b>Adapter / Gateway</b>          | En los Servicios de Integración (Backend).                                      | Este patrón permite que el sistema consuma las APIs externas de la universidad de forma aislada. Cada servicio de integración (ej. <b>ExternalProgramsService</b> ) actúa como un adaptador que transforma la estructura de datos de la API externa al formato interno del sistema. De esta forma, si la API externa cambia, solo se modifica el adaptador, sin afectar al resto del sistema. |
| <b>DTO (Data Transfer Object)</b> | Entre la API (Backend) y el Frontend.                                           | Define la estructura de los datos que se envían a través de la red. Resuelve el problema de la seguridad y la integridad, ya que asegura que solo se transmitan los datos necesarios y en el formato esperado, evitando la exposición de datos sensibles.                                                                                                                                     |
| <b>Inyección de Dependencias</b>  | En toda la arquitectura del Backend (NestJS).                                   | Es un principio fundamental de NestJS que permite que las clases reciban sus dependencias (como servicios o repositorios) desde el exterior. Facilita las pruebas unitarias (ya que se pueden mockear las dependencias) y mejora el desacoplamiento del código.                                                                                                                               |
| <b>Singleton</b>                  | Para el manejo de la configuración global (ej. <b>ConfigService</b> de NestJS). | Asegura que solo exista una instancia de la configuración en toda la aplicación, centralizando el acceso a variables de entorno y otros parámetros. Resuelve el                                                                                                                                                                                                                               |

| Patrón | Dónde se aplica | Problema que resuelve                                     |
|--------|-----------------|-----------------------------------------------------------|
|        |                 | problema de tener configuraciones duplicadas o dispersas. |

► *Patrones habituales en este tipo de proyecto: MVC o MVVM en los clientes, Repository para el acceso a datos, DTO para el transporte entre capas, Inyección de dependencias, Factory, Singleton para configuración. Se evalúa la pertinencia, no la cantidad.*

## 5. Estrategia de despliegue

| Componente            | Dónde se despliega                 | Servicio / proveedor | Costo                                                                                                                                                                          |
|-----------------------|------------------------------------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend / API         | Vercel                             | Vercel               | Gratuito (Plan Hobby). Para un uso moderado, el plan gratuito es suficiente.                                                                                                   |
| Base de datos         | Supabase                           | Supabase             | Gratuito (Plan Free). Proporciona 500 MB de base de datos, suficiente para el desarrollo y pruebas. Durante la migración al servidor de la universidad, el costo pasará a \$0. |
| Frontend web          | Vercel                             | Vercel               | Gratuito (Plan Hobby). Se desplegará como una aplicación Next.js con SSR.                                                                                                      |
| Aplicación móvil      | N/A                                | N/A                  | N/A                                                                                                                                                                            |
| Archivos y multimedia | Supabase S3                        | Supabase S3          | Gratuito (Plan Free). Ofrece 1 GB de almacenamiento, adecuado para la fase de pruebas. Durante la migración al servidor de la universidad (MinIO), el costo pasará a \$0.      |
| Power BI Embedded     | Microsoft Azure / Power BI Service | Power BI Service     | <b>Temporal (Desarrollo):</b><br>Licencia Power BI Pro (versión de prueba gratuita                                                                                             |

| Componente | Dónde se despliega | Servicio / proveedor | Costo                                                                                                                                                                                                                                                                                                               |
|------------|--------------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            |                    |                      | de 60 días) proporcionada por Microsoft.<br><b>Producción (Post-desarrollo):</b> Requiere licencia <b>Microsoft 365 A5</b> o <b>E5</b> (incluye Power BI Pro) para la visualización de reportes incrustados en la web. Esta licencia debe ser gestionada y provista por el equipo administrativo de la universidad. |

► Si el despliegue es sobre infraestructura de la universidad, indique qué dependencia lo autoriza y qué trámite se requiere. Ese trámite suele tardar más de lo previsto: inícielo temprano.

## 6. Estructura del repositorio

Nombre y URL del repositorio, y estrategia de ramas que seguirán.

|                          |                                                                                                                                                                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| URL del repositorio      | <a href="https://github.com/DeibyRamirez/SIAC_Backend">https://github.com/DeibyRamirez/SIAC_Backend</a>                                                                                                                                |
| Estrategia de ramas      | <i>main / develop / feature/*</i>                                                                                                                                                                                                      |
| Convención de commits    | <b>Conventional Commits:</b> Usaremos el estándar para mantener un historial claro y legible.<br><i>Ejemplos:</i><br>feat: add document upload endpoint,<br>fix: resolve date validation bug,<br>docs: update README with setup guide. |
| Tablero de gestión (URL) | <a href="https://app.clickup.com/90171502668/v/l/li/901716407598?pr=90177117506">https://app.clickup.com/90171502668/v/l/li/901716407598?pr=90177117506</a>                                                                            |

## 7. Cronograma preliminar

| Semana | Actividad                                          | Entregable                              | Responsable                                              |
|--------|----------------------------------------------------|-----------------------------------------|----------------------------------------------------------|
| 3–5    | <b>Configuración Inicial e Ingeniería de Datos</b> | 1. Repositorio funcional con CI básico. | <b>Deiby Alejandro Ramirez Galvis</b><br>(Líder Técnico) |

| Semana | Actividad                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | Entregable                                                                                                                                                                                                                                                       | Responsable                                                                                                                                   |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
|        | 1. Configuración de proyectos (NestJS, Next.js, Prisma) con estructura de capas definida.<br>2. Modelado relacional en PostgreSQL (Supabase) y migración inicial.<br>3. Implementación de autenticación (JWT) y control de roles básico (Cargador, Revisor, Administrador).<br>4. Diseño de la estructura de servicios de integración para APIs externas.<br>5. Prototipo de subida y descarga de archivos con <code>@aws-sdk/client-s3</code> a Supabase Storage.<br>6. Configuración de CI/CD básico (GitHub Actions). | 2. Modelo de base de datos definido y migrado.<br>3. Endpoints de autenticación y autorización (login, registro, roles).<br>4. Estructura de servicios de integración definida.<br>5. Prototipo de ingesta de archivos (PDF/Excel) funcionando en entorno local. | <b>David Urrutia Ceron</b> (Apoyo en Testing y Documentación)                                                                                 |
| 6–7    | <b>Desarrollo de Módulos Núcleo (Parte 1)</b><br>1. Biblioteca de plantillas oficiales y control de versiones (descarga, subida, historial).<br>2. Módulo de parseo de Excel (exceljs) para extracción de metadatos y carga a base de datos.<br>3. Implementación del motor de búsqueda y filtros avanzados en Next.js.<br>4. Integración con APIs externas de la universidad (o datos mockeados si no están disponibles).                                                                                               | 1. Módulo de plantillas funcional (CRUD de plantillas).<br>2. Ingesta de datos desde Excel funcionando (parseo y carga).<br>3. Componente de búsqueda y filtros (con estado en URL).<br>4. Conexión exitosa a al menos una API externa (o mock funcional).       | <b>David Urrutia Ceron</b> (Líder de Calidad y Documentación)<br><b>Deiby Alejandro Ramirez Galvis</b> (Apoyo en Backend y Lógica de Negocio) |
| 8–13   | <b>Desarrollo de Módulos Núcleo (Parte 2), Integración y Pruebas de Aceptación</b>                                                                                                                                                                                                                                                                                                                                                                                                                                       | 1. Backend y frontend para gestión de vigencia de documentos.                                                                                                                                                                                                    | <b>Deiby Alejandro Ramirez Galvis</b> (Arquitecto de                                                                                          |

| Semana | Actividad                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Entregable                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Responsable                                                                                                          |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
|        | <p><b>Semanas 8-9:</b></p> <ol style="list-style-type: none"> <li>1. Cálculo de vigencias y lógica de semáforo/contador (● Vigente, ● Próximo, ● Vencido).</li> <li>2. Flujo de aprobación/validación por roles (Cargador → Revisor → Par Académico).</li> <li>3. Sistema de notificaciones in-app.</li> </ol> <p><b>Semanas 10-11:</b></p> <ol style="list-style-type: none"> <li>4. Integración de reportes de Power BI (powerbi-client-react) con licencia de prueba (60 días).</li> <li>5. Configuración de Cron Jobs (node-cron) y sistema de notificaciones por correo (SMTP).</li> <li>6. Desarrollo completo de servicios de integración con APIs externas reales.</li> </ol> <p><b>Semanas 12-13:</b></p> <ol style="list-style-type: none"> <li>7. <b>Migración de credenciales</b> de Supabase al servidor de la universidad (PostgreSQL / MinIO).</li> <li>8. <b>Pruebas de caja negra y funcionales</b> con el cliente (Oscar Alvarado).</li> <li>9. Ajustes finales de UI/UX basados en retroalimentación del cliente.</li> <li>10. Elaboración de manuales de usuario y técnico.</li> </ol> <p><b>Pruebas:</b></p> | <ol style="list-style-type: none"> <li>2. Flujo completo de aprobación funcional (con roles).</li> <li>3. Notificaciones in-app y por correo funcionando.</li> <li>4. Dashboards de Power BI integrados en el frontend.</li> <li>5. Sistema desplegado en el servidor institucional.</li> <li>6. Documento de pruebas de aceptación firmado por el cliente.</li> <li>7. Manuales de usuario y técnico.</li> <li>8. Sistema 100% funcional en producción.</li> </ol> | <p>Software y Backend)</p> <p><b>David Urrutia Ceron</b> (QA, Pruebas, Documentación y Coordinación con Cliente)</p> |

| Semana | Actividad                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Entregable                                                                                                                                                                                                                                                                                                                                                                                     | Responsable                                                                                                                                           |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
|        | <p>- <b>Pruebas de caja negra:</b><br/>Validación de todos los flujos desde la perspectiva del usuario (Cargador, Revisor, Administrador).</p> <p>- <b>Pruebas funcionales:</b><br/>Verificación de que cada módulo cumple con los requisitos establecidos.</p> <p>- <b>Pruebas de integración:</b><br/>Validación de la comunicación con APIs externas, Power BI y servidor SMTP.</p> <p>- <b>Pruebas de migración:</b><br/>Verificación de que los datos y archivos se migran correctamente al servidor institucional.</p>                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                       |
| 14–16  | <p><b>Entrega Final y Presentación</b></p> <ol style="list-style-type: none"> <li>Preparación de la presentación final al docente (PPT + demo en vivo).</li> <li>Ensamble de toda la documentación (técnica, usuario, manuales).</li> <li>Despliegue final a producción (si aplica).</li> <li>Ensayo de la presentación y preparación de respuestas a posibles preguntas.</li> <li>Presentación del proyecto al docente y al cliente.</li> </ol> <p><b>Pruebas:</b> Demostración en vivo de todas las funcionalidades del sistema. Validación final por parte del cliente y del docente.</p> | <ol style="list-style-type: none"> <li>Presentación final (PPT + demo en vivo).</li> <li>Documentación completa (técnica y de usuario) en formato digital.</li> <li>Sistema funcionando en producción (o en entorno de demostración).</li> <li>Carta de satisfacción firmada por el cliente (Oscar Alvarado).</li> <li>Repositorio final con todo el código fuente y documentación.</li> </ol> | <p><b>David Urrutia Ceron</b><br/>(Presentación y Documentación)</p> <p><b>Deiby Alejandro Ramirez Galvis</b><br/>(Demo, Estabilización y Deploy)</p> |

## 8. Aprobación docente

|               |                                                      |
|---------------|------------------------------------------------------|
| Observaciones |                                                      |
| Estado        | <i>Aprobada / Aprobada con ajustes / No aprobada</i> |

---

**Equipo de desarrollo**

Fecha:

---

**Docente**

Fecha: